

03. Arquitectura

AI Dataset Foundry
Versión 0.2.0
2026-09-10
Fuente: 03-architecture.md

03. Arquitectura

Estilo y capas

El proyecto usa una arquitectura de pipeline/puertos ligera. connectors adapta fuentes a DocumentRecord; processors opera sobre texto; exporters y storage materializan resultados; pipeline.build_dataset coordina. CLI, API y escritorio son adaptadores de entrada que reutilizan el mismo núcleo. Las importaciones diferidas aíslan extras opcionales.

```
graph TD
  CLI[Type CLI] --> P[build_dataset]
  API[FastAPI localhost] --> P
  DESK[pywebview + Uvicorn] --> API
  P --> CON[Connector router]
  CON --> MOD[DocumentRecord]
  P --> PROC[Normalize/Clean/Chunk/Dedup/Quality]
  PROC --> CH[ChunkRecord]
  P --> EXP[JSONL/TXT/Parquet]
  P --> DB[(SQLite)]
  P --> MAN[manifest.json]
  AND[Android Java] -. alcance independiente .-> AJ[JSONL offline]
```

No existe contenedor DI, bus de eventos, cola, caché, scheduler ni estado compartido de aplicación. El estado de deduplicación vive durante un build; la UI persiste ejecuciones en work/ui-runs y no implementa limpieza automática.

Secuencia principal

```
sequenceDiagram
  actor User
  participant Entry as CLI/API
  participant Pipe as build_dataset
  participant Conn as ingest
  participant Proc as processors
  participant Out as exporters/SQLite
  User->>Entry: inputs + settings
  Entry->>Pipe: BuildConfig validado
  loop cada locator
    Pipe->>Conn: ingest(locator)
    Conn-->>Pipe: DocumentRecord[] / excepción
  end
  loop cada documento y chunk
    Pipe->>Proc: normalize, clean, chunk, dedup, quality
    Proc-->>Pipe: ChunkRecord aceptado o motivo
  end
  Pipe->>Out: escribir artefactos
  Out-->>Pipe: rutas
  Pipe-->>Entry: registros + manifest
```

Los locators fallidos se aíslan. La escritura de exportadores y SQLite no tiene transacción común: una excepción puede dejar artefactos parciales. SQLite usa la transacción del contexto de conexión y WAL; cada escritura borra/reemplaza el catálogo en esa ruta.

Despliegue

```
flowchart TB
  GH[GitHub main/tag] --> CI[Actions CI/Security]
  GH --> PG[GitHub Pages]
  TAG[Tag v*] --> REL[Workflow Release]
  REL --> WHL[wheel/sdist]
  REL --> WIN[Windows exe]
  REL --> APK[Android APK]
  PC[Equipo local] --> LOOP[127.0.0.1 Uvicorn]
  LOOP --> FS[(work/ + fuentes)]
```

Autenticación/autorización no aplican al modo local confiable y no están implementadas. El control principal es bind loopback; CORS/CSRF explícitos no existen. El frontend y Windows consumen la API local; Android reimplementa un subconjunto, por lo que existe riesgo de divergencia.

Patrones y decisiones

- DTOs/dataclasses para contratos internos; Pydantic para configuración.
- Strategy por funciones para chunking y conectores por extensión.
- IDs estables prefix + SHA-256 truncado; hashes completos para provenance.
- Fail-soft sólo en adquisición y lectura Git de archivos individuales.
- Lazy imports para degradación controlada de extras.
- Síncrono en núcleo; FastAPI deriva el build a thread para no bloquear el event loop.